Import library

```
import pandas as pd  #import csv
import numpy as np  #Data Manipulation
import matplotlib.pyplot as plt  #Visualization
import seaborn as sns  #Visualization
from sklearn import metrics #Import scikit-learn metrics module for accuracy calculation
from sklearn.tree import DecisionTreeClassifier #Decision Tree
from sklearn.model_selection import train_test_split #Train Test spilt
```

Mount Google Drive in Colab

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=Tr
```

Step 1: Data Preprocessing

Step 2: Dummy/Label Encoding

Step 3: Train and Test Split

Step 4: Hyperparameter Tunning

Step 5: Built Decision Tree On Train Data

Step 6: Test the DT Classifier on Testing Data

Step 1 - Data Preprocessing

Import Dataset(CSV File)

```
Housing=pd.read_csv('/content/drive/MyDrive/EDGE/HousingFinance1.csv')
```

Look at Data

```
Housing.head()
```

	ID	Decision	Build_Selfcon	Tier	Accommodation_Class	Marital_Status	Employer_Type	OldEmi_d	BankSave_d	Age	...
0	AJM-39R8-519413	0	Self Contruction	3	0	1	Business	1	0	31	...
1	AJM-3A3Z-876970	0	Self Contruction	3	0	1	Ind/SmallBus	1	0	41	...
2	AJM-7427-952085	1	Self Contruction	3	0	1	Ind/SmallBus	0	0	51	...
3	AJM-8ZTZ-979040	1	Self Contruction	3	1	1	Business	0	0	26	...
4	AJM-9SP2-614810	1	Self Contruction	3	0	1	Business	0	0	31	...

5 rows × 31 columns

Check Missing Value

```
Housing.isna().sum()
```

	0
ID	0
Decision	0
Build_Selfcon	0
Tier	0
Accommodation_Class	0
Marital_Status	0
Employer_Type	0
OldEmi_d	0
BankSave_d	0
Age	0
YrsAdd	0
YrsJob	0
TotInc	0
MarVal	0
LoanReq	0
Term	0
CalcEmi	0
IIR	0
IAR	0
OldEmi	0
LTV	0
FOIR	0

dtype: int64

Summary of dataset

```
Housing.describe()
```

	Decision	Tier	OldEmi_d	BankSave_d	Age	YrsAdd	YrsJob	TotInc	MarVal
count	1570.000000	1570.000000	1570.000000	1570.000000	1570.000000	1570.000000	1570.000000	1570.000000	1.570000e+03
mean	0.799363	1.996815	0.129936	0.119745	37.755414	9.840764	10.929299	29773.247134	1.177865e+06
std	0.400605	0.792325	0.336340	0.324767	8.410670	10.481894	7.986092	18064.471699	6.553948e+05
min	0.000000	1.000000	0.000000	0.000000	22.000000	0.000000	0.000000	7000.000000	5.996000e+04
25%	1.000000	1.000000	0.000000	0.000000	31.000000	2.000000	4.000000	18773.250000	7.897618e+05
50%	1.000000	2.000000	0.000000	0.000000	37.000000	5.000000	10.000000	25127.000000	1.072752e+06
75%	1.000000	3.000000	0.000000	0.000000	43.000000	15.000000	15.000000	35237.500000	1.403509e+06
max	1.000000	3.000000	1.000000	1.000000	63.000000	61.000000	60.000000	219858.000000	7.272727e+06

Step 2- Label Encoding And Dummy Variable

Lable Encoding

```
Housing['Marital_Status'].replace({'Married':1,'Unmarried':0},inplace=True)
Housing['Accommodation_Class'].replace({'Rented':1,'Non_Rented':0},inplace=True)
```

```
/tmp/ipython-input-1212266489.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through ch
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  Housing['Marital_Status'].replace({'Married':1,'Unmarried':0},inplace=True)
/tmp/ipython-input-1212266489.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a
  Housing['Marital_Status'].replace({'Married':1,'Unmarried':0},inplace=True)
/tmp/ipython-input-1212266489.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through ch
```

```
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  Housing['Accommodation_Class'].replace({'Rented':1,'Non_Rented':0},inplace=True)
/tmp/ipython-input-1212266489.py:2: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a
  Housing['Accommodation_Class'].replace({'Rented':1,'Non_Rented':0},inplace=True)
```

Dummy variable

```
Dummy = pd.get_dummies(Housing['Employer_Type'],prefix="Employer_Type",prefix_sep='_').astype(int)
Dummy1 = pd.get_dummies(Housing['Build_Selfcon'],prefix="Build_Selfcon",prefix_sep='_').astype(int)
Dummy2= pd.get_dummies(Housing['Tier'],prefix="Tier",prefix_sep='_').astype(int)
```

Combine All Dummies

```
Housing=pd.concat([Housing,Dummy,Dummy1,Dummy2],axis=1)
```

Step 3 - Test and Train Data

Store Multiple Column Name Together

```
list(Housing.columns)
```

```
['ID',
 'Decision',
 'Build_Selfcon',
 'Tier',
 'Accommodation_Class',
 'Marital_Status',
 'Employer_Type',
 'OldEmi_d',
 'BankSave_d',
 'Age',
 'YrsAdd',
 'YrsJob',
 'TotInc',
 'MarVal',
 'LoanReq',
 'Term',
 'CalcEmi',
 'IIR',
 'IAR',
 'OldEmi',
 'LTV',
 'FOIR',
 'Employer_Type_Business',
 'Employer_Type_Corporate',
 'Employer_Type_Govt',
 'Employer_Type_Ind/SmallBus',
 'Build_Selfcon_Builder',
 'Build_Selfcon_Self Contruction',
 'Tier_1',
 'Tier_2',
 'Tier_3']
```

Feature Selection

```
col=[
 'Accommodation_Class',
 'Marital_Status',
 'OldEmi_d',
 'BankSave_d',
 'Age',
 'YrsAdd',
 'YrsJob',
 'TotInc',
 'MarVal',
 'LoanReq',
 'Term',
 'CalcEmi',
 'IIR',
 'IAR',
 'OldEmi',
 'LTV',
```

```
 'FOIR',
 'Employer_Type_Business',
 'Employer_Type_Corporate',
 'Employer_Type_Govt',
 'Employer_Type_Ind/SmallBus',
 'Build_Selfcon_Builder',
 'Build_Selfcon_Self Contruction',
 'Tier_1',
 'Tier_2',
 'Tier_3',
]
x=Housing[col]
y=Housing.Decision
```

Train And Test Data

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=10)
```

Step 5-Build The Decision Tree On Training Data

```
clf = DecisionTreeClassifier(max_depth=2)
model = clf.fit(x_train,y_train)

y_pred = model.predict(x_test)

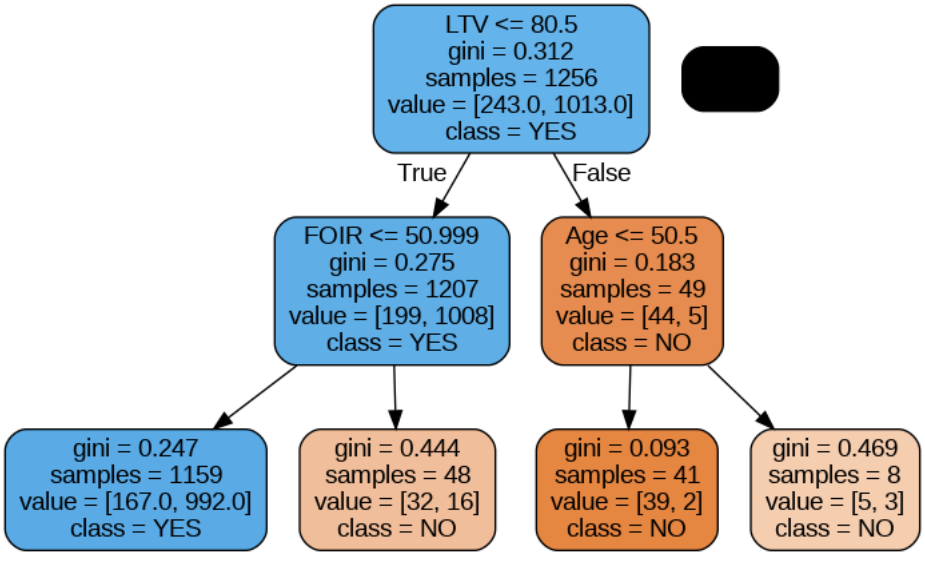
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.8057324840764332
```

Step 6: Test the DT Classifier on Testing Data

```
from pydotplus import graph_from_dot_data
from sklearn.tree import export_graphviz
from IPython.display import Image

dot_data= export_graphviz(clf,filled=True,rounded=True,feature_names=col,class_names=['NO','YES'],out_file=No
graph = graph_from_dot_data(dot_data)
graph.write_png('tree.png')
Image(graph.create_png())
```



Confusion Matrix

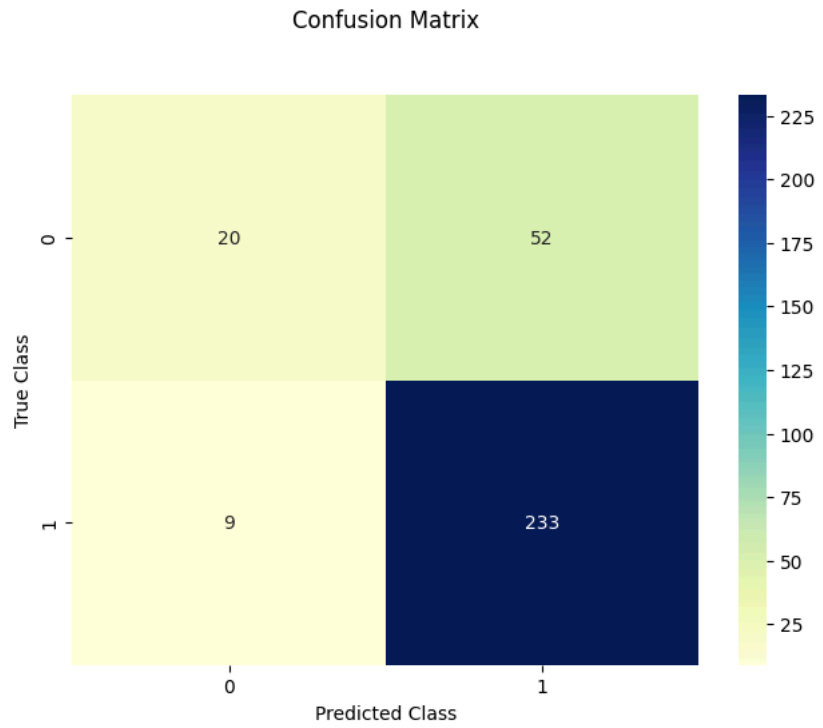
```
cnf_matrix=metrics.confusion_matrix(y_test,y_pred)
cnf_matrix
```

```
array([[ 20,  52],
       [  9, 233]])
```

Heatmap Visualization

```
sns.heatmap(cnf_matrix,annot=True,cmap='YlGnBu',fmt='g')
plt.tight_layout()
plt.ylabel('True Class')
plt.xlabel('Predicted Class')
plt.title('Confusion Matrix',y=1.1)
```

```
Text(0.5, 1.1, 'Confusion Matrix')
```



Step:4 Hyperparameter Tuning

```
paramters={

           "criterion":["entorpy","gini"],
           "max_depth":[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20]
}
```

```
clf1=DecisionTreeClassifier()
```

```
from sklearn.model_selection import GridSearchCV
grid=GridSearchCV(estimator=clf1,param_grid=paramters,cv=10)
```

```
grid.fit(x_train,y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.py:528: FitFailedWarning:
200 fits failed out of a total of 400.
The score on these train-test partitions for these parameters will be set to nan.
If these failures are not expected, you can try to debug them by setting error_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
200 fits failed with the following error:
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1382, in wrapper
    estimator._validate_params()
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 436, in _validate_params
    validate_parameter_constraints(
  File "/usr/local/lib/python3.12/dist-packages/sklearn/utils/_param_validation.py", line 98, in validate_parameter_constrai
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'criterion' parameter of DecisionTreeClassifier must be a str amc

  warnings.warn(some_fits_failed_message, FitFailedWarning)
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_search.py:1108: UserWarning: One or more of the test scores
       nan        nan        nan        nan        nan        nan
       nan        nan        nan        nan        nan        nan
       nan        nan 0.8367619  0.84866667 0.83993651 0.82165079
 0.81846349 0.81526984 0.81130159 0.81053333 0.79539048 0.78500952
 0.78425397 0.77789206 0.77313016 0.77788571 0.7627746  0.76516825
 0.76833651 0.76992381 0.77073016 0.76593016]
  warnings.warn(
```

▸ **GridSearchCV** ⓘ ?

▸ **best_estimator_: DecisionTreeClassifier**

▸ DecisionTreeClassifier ?

```
grid.best_score_
```

```
np.float64(0.8486666666666668)
```